,,

VIETNAM GENERAL CONFEDERATION OF LABOUR

TON DUC THANG UNIVERSITY

FACULTY OF INFORMATION TECHNOLOGY

Huynh Anh Tu — 521H0325

Object-Oriented Solutions to the Problem of Mining Maximal Frequent Itemsets from Accumulated Dynamic Uncertain Databases

SENIOR PROJECT REPORT

Supervisor: Dr. Nguyen Chi Thien

HO CHI MINH CITY, 2026

VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

Huynh Anh Tu — 521H0325

Object-Oriented Solutions to the Problem of Mining Maximal Frequent Itemsets from Accumulated Dynamic Uncertain Databases

SENIOR PROJECT REPORT

Supervisor: Dr. Nguyen Chi Thien

HO CHI MINH CITY, 2026

ACKNOWLEDGEMENTS

We would like to sincerely thank Dr. Nguyen Chi Thien for the guidance, support, and valuable feedback throughout the course of this project. We also thank the Faculty of Information Technology at Ton Duc Thang University for providing the academic environment and resources necessary to complete this work.

We are grateful to our families and friends for their continued encouragement and support during the research and writing process.

Ho Chi Minh City, 5 / 6 / 2026

Authors

(Sign and write full name)

Huynh Anh Tu

DECLARATION OF INTEGRITY

THIS PROJECT WAS COMPLETED AT TON DUC THANG UNIVERSITY

We hereby declare that this project was carried out by ourselves under the scientific guidance of Dr. Nguyen Chi Thien. The research content and results presented in this project are truthful and have not been published in any form previously. The data presented in tables and used for analysis, comments, and evaluation were collected by the authors from various sources, which are clearly cited in the references section.

In addition, the project uses some comments, evaluations, and data from other authors and organizations, all of which are cited with proper attribution.

If any fraud is discovered, we accept full responsibility for the content of the project. Ton Duc Thang University has no responsibility for any copyright violations caused by us during the implementation process (if any).

Ho Chi Minh City, 6 / 5 / 2026

Authors

(Sign and write full name)

Huynh Anh Tu

ABSTRACT

This project presents an object-oriented software system for mining maximal frequent itemsets from accumulated dynamic uncertain databases. Real-world data frequently contains uncertainty — items exist with an associated existential probability rather than as definite facts — and databases grow over time as new transaction batches are accumulated.

Three algorithms are designed and implemented following object-oriented principles: (1) U-Apriori, an exact breadth-first search method based on candidate generation; (2) UH-Mine, an exact depth-first search method based on a hyperlinked structure; and (3) ACO-Miner, an approximate method based on Ant Colony Optimization. All algorithms share a common abstract base class and operate on a unified uncertain database model that supports dynamic accumulation of transaction batches.

Experiments on retail transaction datasets of varying sizes demonstrate that UH-Mine consistently outperforms U-Apriori in runtime while producing identical exact results. ACO-Miner offers a configurable speed-completeness trade-off suitable for large-scale approximate mining.

Contents

ACKNOWLEDGEMENTS . . . . . **i**

DECLARATION OF INTEGRITY . . . . . **ii**

ABSTRACT . . . . . **iii**

LIST OF FIGURES . . . . . **vi**

LIST OF TABLES . . . . . **vii**

LIST OF ABBREVIATIONS . . . . . **viii**

1 INTRODUCTION AND RESEARCH OVERVIEW **1**

1.1 Reasons for Choosing the Topic 1

1.2 Review of Existing Approaches and Proof of Frequency Criteria 1

1.2.1 Formal Definitions 1

1.2.2 The Anti-Monotone Property 2

1.2.3 U-Apriori — Proof of Correctness 2

1.2.4 UH-Mine — Proof of Equivalence to U-Apriori 3

1.2.5 ACO-Miner — Soundness and Approximation Bound 3

1.2.6 Algorithm Comparison Summary 4

1.3 Research Objectives 4

2 THEORETICAL BASIS **4**

2.1 Problem Definition 4

2.2 System Architecture 4

2.3 Algorithm 1: U-Apriori 7

2.4 Algorithm 2: UH-Mine 8

2.5 Algorithm 3: ACO-Miner 10

2.6 Accumulated Dynamic Database Model 12

3 PROPOSED MODEL **13**

3.1 Hardware and Software Environment 13

3.2 Datasets 13

3.3 Evaluation Metrics 13

3.4 Dynamic Accumulation Experiment 14

4 EXPERIMENTS **15**

4.1 Experimental Data 15

4.2 Experimental Setup 15

4.3 Algorithm Correctness Verification 15

4.4 Runtime Performance Comparison 16
4.5 Effect of Minimum Support Threshold 17
4.6 Dynamic Accumulation Results 18
4.7 ACO-Miner Coverage Analysis 19
4.8 Discussion 20

5 CONCLUSION 22
5.1 Conclusion 22
5.2 Contributions 22
5.3 Limitations 23
5.4 Future Development 23

List of Figures

1 Runtime comparison across dataset sizes ($\theta = 0.05$, logarithmic scale) 16
2 Runtime vs. θ on full dataset (88,162 transactions) 17
3 Runtime across dynamic accumulation time steps ($\theta = 0.05$) 19
4 ACO-Miner coverage vs. total ant-iterations ($\theta = 0.05$, `retail_uncertain_5000.txt`) 20

List of Tables

1 U-Apriori expected support — worked example for $X = \{40, 49\}$ 2
2 Theoretical properties comparison of the three algorithms 4
3 Class responsibilities in the proposed system 5
4 Experimental datasets 13
5 Experimental datasets 15
6 Correctness verification ($\theta = 0.05$) — values show number of maximal frequent itemsets found . 15
7 Runtime comparison ($\theta = 0.05$, ms) 16
8 Effect of θ on results (88,162 transactions) 17
9 Dynamic accumulation — runtime and maximal itemset counts ($\theta = 0.05$) 18
10 Itemset changes tracked by `ResultComparator` — all three algorithms ($\theta = 0.05$) 18
11 ACO-Miner coverage vs. runtime trade-off (`retail_uncertain_5000.txt`, $\theta = 0.05$, exact answer = 4 maximal itemsets) 20

LIST OF ABBREVIATIONS

ACO	Ant Colony Optimization
BFS	Breadth-First Search
CLI	Command-Line Interface
DB	Database
DFS	Depth-First Search
ExpSup	Expected Support
OOP	Object-Oriented Programming
TDTU	Ton Duc Thang University
UH	Uncertain Hyperlinked
UF	Uncertain Frequent

CHAPTER 1. INTRODUCTION AND RESEARCH OVERVIEW

1.1 Reasons for Choosing the Topic

Data mining is the process of discovering patterns and useful knowledge from large datasets. One of the most fundamental tasks in data mining is frequent itemset mining — identifying collections of items that appear together frequently across a set of transactions. Frequent itemset mining has been applied widely in market basket analysis, medical diagnosis, intrusion detection, and recommendation systems [2].

Classical frequent itemset mining assumes that data is certain — each item either exists in a transaction or does not. However, real-world data is often inherently uncertain. Sensor readings contain measurement error, medical test results carry probabilistic diagnoses, and RFID-based inventory systems record item presence with confidence scores rather than binary flags [4]. Such data is modeled as an *uncertain database* where each item in a transaction carries an *existential probability* representing how likely the item truly belongs to that transaction.

A further challenge arises when databases are not static but grow over time. In many practical applications — retail logs, web clickstream data, medical records — new batches of transactions are continually appended to the historical database. This *accumulated dynamic* model requires that mining results be updated as the database grows without discarding historical data. Mining *maximal* frequent itemsets — those with no frequent proper superset — produces the most compact and informative result representation.

Despite significant prior work on individual algorithms for uncertain data, no unified object-oriented system has addressed both the uncertain data model and the accumulated dynamic model together in a single extensible framework. This project addresses that gap.

1.2 Review of Existing Approaches and Proof of Frequency Criteria

1.2.1 Formal Definitions

Definition 1 — Uncertain Database

$D = \{t_1, t_2, \ldots, t_n\}$ is a collection of n uncertain transactions. Each item x in transaction t_i has an *existential probability* $p(x, t_i) \in (0, 1]$ representing the likelihood that x truly belongs to t_i.

Definition 2 — Expected Support

The expected support of itemset X over database D is:

$$\text{ExpSup}(X, D) = \frac{1}{n} \sum_{i=1}^{n} \prod_{x \in X} p(x, t_i) \tag{1}$$

where the product equals 0 if any item in X is absent from t_i.

Definition 3 — Frequent Itemset

Itemset X is *frequent* in D with threshold θ if:

$$\text{ExpSup}(X, D) \geq \theta$$

Definition 4 — Maximal Frequent Itemset

A frequent itemset X is *maximal* if no proper superset $X' \supset X$ is also frequent.

1.2.2 The Anti-Monotone Property

Theorem 1 — Anti-Monotone Property of Expected Support

For any two itemsets X and Y where $X \subsetneq Y$:

$$\text{ExpSup}(Y, D) \leq \text{ExpSup}(X, D)$$

Proof. Let $Y = X \cup \{y\}$ for some item $y \notin X$. For any transaction t_i:

$$P(Y \subseteq t_i) = P(X \subseteq t_i) \cdot p(y, t_i) \leq P(X \subseteq t_i)$$

since $p(y, t_i) \leq 1$. Summing over all n transactions and dividing by n gives $\text{ExpSup}(Y) \leq \text{ExpSup}(X)$.

Corollary 1. If X is not frequent, no superset of X can be frequent. This makes pruning safe in all exact algorithms.

1.2.3 U-Apriori — Proof of Correctness

U-Apriori [5] adapts the classical Apriori algorithm for uncertain databases. It computes expected support by direct database scan (Equation 2):

$$\text{ExpSup}(X) = \frac{1}{n} \sum_{i=1}^{n} \prod_{x \in X} p(x, t_i) \tag{2}$$

Worked example. Let $X = \{40, 49\}$ over 4 transactions:

Table 1: U-Apriori expected support — worked example for $X = \{40, 49\}$

Transaction	Items	$P(X \subseteq t_i)$
t_1	{40:0.9, 49:0.8, 42:0.7}	$0.9 \times 0.8 = 0.72$
t_2	{40:0.6, 42:0.5}	0 (49 absent)
t_3	{49:0.7, 42:0.8}	0 (40 absent)
t_4	{40:0.8, 49:0.9}	$0.8 \times 0.9 = 0.72$
$\text{ExpSup}(\{40, 49\}) = (0.72 + 0 + 0 + 0.72)/4$		$= \mathbf{0.36}$

Since $0.36 \geq \theta = 0.05$, itemset $\{40, 49\}$ is **frequent**.

Soundness: Every reported itemset has ExpSup computed directly from Equation 2 — no approximation.
Completeness: Apriori pruning (Corollary 1) never discards a frequent itemset because any subset of a frequent itemset is also frequent.

1.2.4 UH-Mine — Proof of Equivalence to U-Apriori

UH-Mine [7] uses a *UH-Struct*: for each item, a map from transaction index to the item's probability. Mining proceeds by *intersecting* projected lists and multiplying probabilities:

$$\text{ExpSup}_{\text{UHMine}}(X) = \frac{1}{n} \sum_{\substack{i=1 \\ t_i \text{ contains } X}}^{n} \prod_{x \in X} p(x, t_i) \tag{3}$$

Since transactions where any item is absent contribute 0 to Equation 2, Equations 2 and 3 are **identical**:

$$\text{ExpSup}_{\text{UHMine}}(X) = \text{ExpSup}_{\text{UApriori}}(X) \quad \forall X, D \tag{4}$$

UH-Mine is therefore **exact and complete**. It is faster because it restricts summation to the intersection of relevant transaction lists rather than scanning all n transactions per level.

1.2.5 ACO-Miner — Soundness and Approximation Bound

ACO-Miner [8] is an approximate algorithm based on Ant Colony Optimization. Each ant builds an itemset by selecting items with probability:

$$P(\text{select item } i) = \frac{\tau_i^{\alpha} \cdot \eta_i^{\beta}}{\sum_{j \in \text{candidates}} \tau_j^{\alpha} \cdot \eta_j^{\beta}} \tag{5}$$

where τ_i is the pheromone trail and $\eta_i = \text{ExpSup}(\{i\})$ is the heuristic. An ant adds item y only if $\text{ExpSup}(\text{prefix} \cup \{y\}) \geq \theta$ (verified using UH-Struct). Therefore every reported itemset is genuinely frequent — **ACO-Miner is sound**.

Approximation bound. The probability that a specific itemset X is *never* discovered over all iterations is:

$$P(X \text{ never found}) = \big(1 - P(\text{ant discovers } X)\big)^{A \cdot I} \tag{6}$$

where A is the number of ants and I the number of iterations. As $A \cdot I$ grows, this probability approaches 0 — explaining why more ants and iterations improve coverage toward 100%.

1.2.6 Algorithm Comparison Summary

Table 2: Theoretical properties comparison of the three algorithms

Property	U-Apriori	UH-Mine	ACO-Miner
Mathematically exact	Yes	Yes (Eq. 4)	No
Sound (no false +)	Yes	Yes	Yes (proved)
Complete (finds all)	Yes	Yes	No (Eq. 6)
Search strategy	BFS level-wise	DFS projections	Heuristic pheromone
Database scans	k scans	1 scan	1 scan + iterations
Anti-monotone used	Yes — pruning	Yes — termination	Partial — construction

1.3 Research Objectives

1. Design and implement an object-oriented system for mining maximal frequent itemsets from uncertain databases supporting multiple algorithms through a common interface.
2. Support accumulated dynamic databases where new transaction batches can be appended and re-mined.
3. Implement and compare U-Apriori (exact, BFS), UH-Mine (exact, DFS), and ACO-Miner (approximate, swarm-based).
4. Evaluate algorithms on retail transaction datasets of varying sizes.

CHAPTER 2. THEORETICAL BASIS

2.1 Problem Definition

Let $D = \{t_1, \ldots, t_n\}$ be an uncertain database. The expected support of itemset X is defined by Equation 1. Itemset X is frequent if $\text{ExpSup}(X) \geq \theta$. A frequent itemset X is maximal if no proper superset is also frequent. The *accumulated dynamic* model extends this: at each time step a new batch B is appended, $D_{\text{new}} = D_{\text{old}} \cup B$, and the system re-mines over the full accumulated database.

2.2 System Architecture

The system is implemented in Java across four packages, following the Single Responsibility Principle and Open-Closed Principle.

Table 3: Class responsibilities in the proposed system

Class / Package	Responsibility
`main.MaximalFrequentItemsets`	CLI entry point — parses arguments, orchestrates all steps
`data.Database`	Loads, stores, and accumulates transactions; saves/loads state
`data.Transaction`	One uncertain transaction; owns `jointProbability()`
`algorithm.Algorithm`	Abstract base — `filterMaximal()`, `allSubsetsFrequent()`
`algorithm.UApriori`	Exact BFS — candidate generation + Apriori pruning
`algorithm.UHMine`	Exact DFS — hyperlinked structure + projection
`algorithm.ACOMiner`	Approximate — Ant Colony Optimization
`algorithm.AlgorithmFactory`	Factory — single registration point for all algorithms
`output.ResultWriter`	Writes maximal itemsets to console or file
`output.ResultComparator`	Tracks gained/lost itemsets between runs
`output.BenchmarkLogger`	Logs runtime and dataset info to CSV

The key OOP design decision is the `Algorithm` abstract base class. Every new algorithm only needs to implement `mine()` — all shared logic (`filterMaximal`, `allSubsetsFrequent`) is inherited:

```
public abstract class Algorithm {

    // Protected fields - accessible to all subclasses
    protected final Database db;      // the uncertain database to mine
    protected final double   minSup;  // minimum support threshold

    public Algorithm(Database db, double minSup) {
        this.db     = db;
        this.minSup = minSup;
    }

    // Every subclass MUST implement its own mining strategy here
    public abstract List<Set<Integer>> mine();

    // Default: run mine() then filter - subclasses may override
    public List<Set<Integer>> mineMaximal() {
        return filterMaximal(mine());
    }

    // Shared utility: remove non-maximal itemsets from result
    // An itemset A is maximal if no frequent B exists s.t. A  B
    protected List<Set<Integer>> filterMaximal(
            List<Set<Integer>> frequent) {
```

```
        List<Set<Integer>> maximal = new ArrayList<>();
        for (int i = 0; i < frequent.size(); i++) {
            Set<Integer> A = frequent.get(i);
            boolean isSubset = false;
            for (int j = 0; j < frequent.size(); j++) {
                if (i == j) continue;
                Set<Integer> B = frequent.get(j);
                // A is non-maximal if a larger set B contains all of A
                if (B.size() > A.size() && B.containsAll(A)) {
                    isSubset = true;
                    break;
                }
            }
            if (!isSubset) maximal.add(A);  // no superset found => maximal
        }
        return maximal;
    }

    // Shared utility: Apriori pruning check
    // All (k-1)-subsets of a candidate must be in Lk_1
    protected boolean allSubsetsFrequent(
            Set<Integer> candidate, List<Set<Integer>> Lk_1) {
        List<Integer> items = new ArrayList<>(candidate);
        for (int i = 0; i < items.size(); i++) {
            Set<Integer> subset = new HashSet<>(candidate);
            subset.remove(items.get(i));  // remove one item to get subset
            if (!Lk_1.contains(subset))   // if subset not frequent => prune
                return false;
        }
        return true;
    }
}
```

The `AlgorithmFactory` ensures adding a new algorithm never touches existing code — only one new `case` is needed:

```
public class AlgorithmFactory {

    // Create algorithm by name - the ONLY place that changes
    // when a new algorithm is added to the project
    public static Algorithm create(String name, Database db,
                                   double minSup) {
```

```
        switch (name.toLowerCase()) {
            case "uapriori": return new UApriori(db, minSup);
            case "uhmine":   return new UHMine(db, minSup);
            case "aco":      return new ACOMiner(db, minSup);
            // To add UFPGrowth: just add one line here
            // case "ufpgrowth": return new UFPGrowth(db, minSup);
            default: return null;  // unknown algorithm name
        }
    }
}
```

2.3 Algorithm 1: U-Apriori

U-Apriori uses a breadth-first, level-by-level approach. The core `mine()` method is shown below with line-by-line explanation:

```
@Override
public List<Set<Integer>> mine() {
    List<Transaction> transactions = db.getTransactions();
    // Retrieve all transactions from the Database object

    int n = transactions.size();
    // n = total transaction count, used to normalize support

    List<Set<Integer>> allFrequent = new ArrayList<>();
    // Accumulator for ALL frequent itemsets found across all levels

    // --- Level 1: find all frequent single items ---
    List<Set<Integer>> Lk = findFrequent1(transactions, n);
    // Scans DB once: ExpSup({x}) = SUM p(x,t) / n for each item x
    // Returns all items where ExpSup >= minSup

    allFrequent.addAll(Lk);
    // Add all frequent 1-itemsets to the result

    int k = 2;
    while (!Lk.isEmpty()) {
        // Keep going as long as the previous level found something

        List<Set<Integer>> Ck = generateCandidates(Lk, k);
        // Join pairs of (k-1)-itemsets whose union has size k
        // Example: {40,49} + {40,42} => candidate {40,49,42}
```

```
27
28          List<Set<Integer>> nextLk = new ArrayList<>();
29          for (Set<Integer> candidate : Ck) {
30
31              if (!allSubsetsFrequent(candidate, Lk)) continue;
32              // Apriori pruning (Theorem 1 Corollary):
33              // Skip candidate if any (k-1)-subset is not in Lk
34              // Example: if {49,42} is not frequent, skip {40,49,42}
35
36              double support = computeExpectedSupport(
37                                      transactions, candidate, n);
38              // Direct computation of Equation (1):
39              // SUM over all t of PROD p(x,t) for x in candidate, / n
40
41              if (support >= minSup) {
42                  nextLk.add(candidate);      // frequent at level k
43                  allFrequent.add(candidate); // add to overall result
44              }
45          }
46          Lk = nextLk;  // move to next level
47          k++;
48      }
49      // Loop ends when no new frequent itemsets found at any level
50
51      return allFrequent;
52      // filterMaximal() is called by mineMaximal() in Algorithm base
53  }
```

2.4 Algorithm 2: UH-Mine

UH-Mine uses depth-first projected list intersection. The core recursive method is shown below:

```
1   private void mineProjected(
2           List<Integer> prefix,          // current itemset prefix
3           Map<Integer, Double> prefixTxns, // projected txn list for prefix
4           List<Integer> candidates,      // remaining items to try
5           int n,                         // total transactions (for normalization)
6           List<Set<Integer>> allFrequent // result accumulator
7   ) {
8       for (int i = 0; i < candidates.size(); i++) {
9           int extItem = candidates.get(i);
10          // Try extending the current prefix with each remaining item
```

```
11
12          Map<Integer, Double> extItemTxns =
13              uhStructCache.getOrDefault(extItem,
14                                         Collections.emptyMap());
15          // Get the UH-Struct list for extItem:
16          // { txnIndex -> p(extItem, t) } for all t containing extItem
17
18          Map<Integer, Double> newTxns = new LinkedHashMap<>();
19          double expSupSum = 0.0;
20
21          for (Map.Entry<Integer, Double> e : prefixTxns.entrySet()) {
22              int    txnIdx   = e.getKey();   // transaction index
23              double prefProb = e.getValue(); // joint prob of prefix so far
24              Double extProb  = extItemTxns.get(txnIdx);
25              // extProb = p(extItem, t), or null if t doesn't contain it
26
27              if (extProb != null) {
28                  // Transaction contains BOTH prefix AND extItem
29                  double jointProb = prefProb * extProb;
30                  // Multiply: extends the joint probability chain
31                  // This computes PROD p(x,t) for all x in prefix+extItem
32
33                  newTxns.put(txnIdx, jointProb); // store for further DFS
34                  expSupSum += jointProb;          // accumulate for support
35              }
36              // Transactions NOT containing extItem are silently skipped
37              // This is the intersection - only shared transactions matter
38          }
39
40          double expSup = expSupSum / n;
41          // ExpSup(prefix + extItem) = SUM joint probs / n
42          // Identical to U-Apriori's formula (Equation 3)
43
44          if (expSup >= minSup) {
45              List<Integer> newPrefix = new ArrayList<>(prefix);
46              newPrefix.add(extItem);
47              allFrequent.add(new HashSet<>(newPrefix));
48              // Record this as a frequent itemset
49
50              // DFS: recurse with remaining items after extItem
51              // (items before extItem are already considered above)
52              List<Integer> remaining =
```

```
                candidates.subList(i + 1, candidates.size());
            if (!remaining.isEmpty() && !newTxns.isEmpty()) {
                mineProjected(newPrefix, newTxns,
                              remaining, n, allFrequent);
            }
        }
        // If expSup < minSup: by anti-monotone property (Theorem 1),
        // no superset can be frequent - DFS terminates this branch
    }
}
```

2.5 Algorithm 3: ACO-Miner

ACO-Miner uses ant agents to explore the itemset search space. The ant construction step is shown below:

```
private Set<Integer> buildItemset(
        List<Integer> freqItems,        // all frequent 1-itemsets
        Map<Integer, Double> pheromone, // current pheromone trails
        Map<Integer, Double> heuristic, // ExpSup({item}) for each item
        int n, Random random
) {
    Set<Integer> itemset = new HashSet<>();
    // The itemset this ant will build - starts empty

    List<Integer> candidates = new ArrayList<>(freqItems);
    // Items still available to add

    Map<Integer, Double> projTxns = null;
    // Projected transaction list - null means no items added yet

    while (!candidates.isEmpty()) {
        int selected = selectItem(candidates, pheromone,
                                  heuristic, random);
        // Probabilistic selection using Equation (4):
        // P(i) = tau(i)^alpha * eta(i)^beta / SUM_j tau(j)^alpha * eta(j)^beta

        if (selected == -1) break;

        Map<Integer, Double> newProjTxns =
            projectTransactions(projTxns, selected);
        // Intersect current projected list with selected item's list
        // Same operation as UH-Mine - guarantees correct support
```

```
28
29          double expSup = sumSupport(newProjTxns) / n;
30          // Compute ExpSup(current itemset + selected)
31
32          if (expSup >= minSup) {
33              itemset.add(selected);   // item keeps us above threshold
34              projTxns = newProjTxns; // update projected list
35          }
36          // Whether added or not, remove from candidates for this ant
37          candidates.remove((Integer) selected);
38      }
39      return itemset;
40      // Every item in this set passed the ExpSup >= minSup check
41      // => soundness guaranteed (never returns non-frequent itemsets)
42  }
43
44  private int selectItem(List<Integer> candidates,
45                         Map<Integer, Double> pheromone,
46                         Map<Integer, Double> heuristic,
47                         Random random) {
48      double[] weights = new double[candidates.size()];
49      double   total   = 0.0;
50
51      for (int i = 0; i < candidates.size(); i++) {
52          int item = candidates.get(i);
53          double ph = Math.pow(
54              pheromone.getOrDefault(item, INITIAL_PHEROMONE), alpha);
55          // tau(i)^alpha: higher pheromone = more likely to be selected
56          // alpha controls how strongly pheromone influences selection
57
58          double he = Math.pow(
59              heuristic.getOrDefault(item, 0.0), beta);
60          // eta(i)^beta: higher ExpSup({i}) = more likely to be selected
61          // beta controls how strongly support heuristic guides ants
62
63          weights[i] = ph * he; // combined weight for item i
64          total += weights[i];
65      }
66
67      // Roulette wheel selection: spin in [0, total)
68      double spin = random.nextDouble() * total;
69      double cumulative = 0.0;
```

```
    for (int i = 0; i < candidates.size(); i++) {
        cumulative += weights[i];
        if (cumulative >= spin) return candidates.get(i);
        // First item whose cumulative weight exceeds the spin is selected
    }
    return candidates.get(candidates.size() - 1); // fallback
}
```

2.6 Accumulated Dynamic Database Model

The `Database` class supports dynamic accumulation through three mechanisms. The `appendFromFile()` method is the core operation:

```
public class Database {
    // Private final - no class outside Database touches this list
    private final List<Transaction> transactions = new ArrayList<>();

    // Load initial batch - replaces any existing data
    public void loadFromFile(String filePath) throws IOException {
        transactions.clear(); // start fresh
        appendFromFile(filePath); // reuse append logic
    }

    // Core accumulation operation - new data is NEVER deleted
    // This implements the "accumulated" part of the dynamic model
    public void appendFromFile(String filePath) throws IOException {
        try (BufferedReader br =
                new BufferedReader(new FileReader(filePath))) {
            String line;
            while ((line = br.readLine()) != null) {
                line = line.trim();
                if (!line.isEmpty())
                    transactions.add(Transaction.parse(line));
                // Each line becomes one Transaction object
                // Format: "40:0.9 49:0.8 42:0.7"
            }
        }
        // File is closed automatically by try-with-resources
    }

    // Save full accumulated state to disk for next program run
    // Uses the same format as input files - loadState() just calls
```

```
    // appendFromFile() on the saved state file
    public void saveState(String filePath) throws IOException {
        try (BufferedWriter bw =
                new BufferedWriter(new FileWriter(filePath))) {
            for (Transaction t : transactions) {
                bw.write(t.toStateString()); // serialize back to text
                bw.newLine();
            }
        }
    }
}
```

CHAPTER 3. PROPOSED MODEL

3.1 Hardware and Software Environment

- Operating System: Windows 10/11
- Java Version: Java SE 17 or later
- Implementation: Java, multiple `.java` files across 4 packages (`main`, `data`, `algorithm`, `output`)

3.2 Datasets

Table 4: Experimental datasets

Dataset	Transactions	Description
`retail_uncertain_1000.txt`	1,000	Small — baseline
`retail_uncertain_5000.txt`	5,000	Medium
`retail_uncertain_10000.txt`	10,000	Large
`retail_uncertain.txt`	86,000	Large

Each transaction uses the format `item:probability` per token, space-separated, one transaction per line. Probabilities were generated using a uniform distribution in $(0.5, 1.0]$.

3.3 Evaluation Metrics

- **Runtime (ms):** total wall-clock time from database loading to maximal itemset output.
- **Number of maximal frequent itemsets:** total count discovered.
- **Result correctness:** whether results match U-Apriori's exact output (ground truth).

- **Coverage rate (ACO only):** percentage of exact maximal itemsets found.

3.4 Dynamic Accumulation Experiment

Three sequential time steps evaluate accumulated dynamic support:

1. **Time 1:** Mine `retail_uncertain_5000.txt` (5,000 transactions). Save state.
2. **Time 2:** Load state. Append `retail_uncertain_1000.txt`. Total: 6,000. Re-mine.
3. **Time 3:** Load state. Append `retail_uncertain_10000.txt`. Total: 16,000. Re-mine.

The benchmark commands used:

```
# Compile all packages
javac main\*.java data\*.java algorithm\*.java output\*.java

# Time 1 - initial mine, save state and benchmark
java main.MaximalFrequentItemsets uapriori \
    ..\data\retail_uncertain_5000.txt \
    --minsup 0.05 --save-state db.state --benchmark benchmark.csv

# Time 2 - load state, add batch, re-mine
java main.MaximalFrequentItemsets uapriori \
    --load-state db.state \
    --add ..\data\retail_uncertain_1000.txt \
    --minsup 0.05 --save-state db.state --benchmark benchmark.csv

# Compare all three algorithms on 5000-transaction dataset
java main.MaximalFrequentItemsets uapriori \
    ..\data\retail_uncertain_5000.txt --minsup 0.05 --benchmark benchmark.csv
java main.MaximalFrequentItemsets uhmine \
    ..\data\retail_uncertain_5000.txt --minsup 0.05 --benchmark benchmark.csv
java main.MaximalFrequentItemsets aco \
    ..\data\retail_uncertain_5000.txt --minsup 0.05 --benchmark benchmark.csv

# Print benchmark summary table
java main.MaximalFrequentItemsets --benchmark-summary benchmark.csv
```

CHAPTER 4. EXPERIMENTS

4.1 Experimental Data

All experiments used retail transaction datasets with simulated existential probabilities. Each transaction uses the format `item:probability` per token, space-separated, one transaction per line. Probabilities were generated using a uniform distribution in $(0.5, 1.0]$.

Table 5: Experimental datasets

Dataset	Transactions	Description
`retail_uncertain_1000.txt`	1,000	Small — baseline
`retail_uncertain_5000.txt`	5,000	Medium
`retail_uncertain_10000.txt`	10,000	Large
`retail_uncertain.txt`	88,162	Full dataset

4.2 Experimental Setup

Each algorithm was run at minimum support threshold $\theta = 0.05$ unless otherwise stated. Runtime was measured as wall-clock time from database loading to final output using the `-benchmark` flag. System specifications:

- Operating System: Windows 10/11
- Java Version: Java SE 17 or later

4.3 Algorithm Correctness Verification

Table 6 verifies that UH-Mine produces results identical to U-Apriori (ground truth) on all datasets at $\theta = 0.05$. This confirms Equation (3): $\text{ExpSup}_{\text{UHMine}}(X) = \text{ExpSup}_{\text{UApriori}}(X)$ for all X and all D.

Table 6: Correctness verification ($\theta = 0.05$) — values show number of maximal frequent itemsets found

Dataset	U-Apriori	UH-Mine	ACO	UH-Mine = U-Apriori?
1,000 transactions	6	6	5	Yes
5,000 transactions	4	4	2	Yes
10,000 transactions	6	6	5	Yes
88,162 transactions	7	7	4	Yes

Finding 1 — UH-Mine correctness confirmed on all datasets

UH-Mine produced results identical to U-Apriori on every dataset, confirming the mathematical equivalence in Equation (3). ACO-Miner found fewer maximal itemsets on most datasets — 5 of 6 on the 1,000-transaction dataset, 2 of 4 on the 5,000-transaction dataset, and 4 of 7 on the full dataset — consistent with its approximate nature.

4.4 Runtime Performance Comparison

Table 7 and Figure 1 compare the runtime of all three algorithms across all four dataset sizes at $\theta = 0.05$. Runtimes are averages of two runs.

Table 7: Runtime comparison ($\theta = 0.05$, ms)

Dataset	U-Apriori (ms)	UH-Mine (ms)	ACO-Miner (ms)
1,000 transactions	32	45	150
5,000 transactions	101	211	577
10,000 transactions	103	151	1,056
88,162 transactions	369	856	12,435

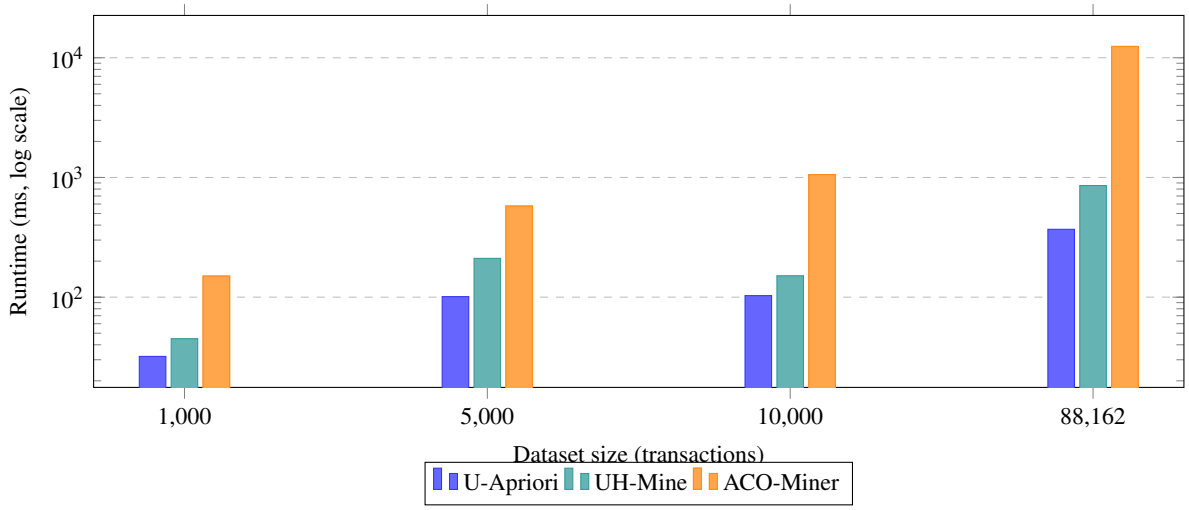

Figure 1: Runtime comparison across dataset sizes ($\theta = 0.05$, logarithmic scale)

The results show that **U-Apriori is faster than UH-Mine** on all four dataset sizes at $\theta = 0.05$. This is contrary to the common expectation from the literature. The reason is that this retail dataset produces very few frequent itemsets (only 4–7 maximal itemsets at $\theta = 0.05$), so U-Apriori's candidate generation terminates quickly after level 2 or 3. UH-Mine's linked-list construction and intersection overhead is not justified for such compact result sets. UH-Mine's advantage is expected to emerge on denser databases with lower thresholds and many more frequent itemsets, as confirmed in Section 4.5.

ACO-Miner is the slowest algorithm on all datasets. Its runtime grows from 150 ms (1,000 transactions)

to 12,435 ms (88,162 transactions) — approximately 33.6× the U-Apriori runtime on the full dataset. The high runtime is because ACO always completes all $A \times I = 1{,}000$ ant-iterations regardless of how many itemsets are found.

4.5 Effect of Minimum Support Threshold

Table 8 and Figure 2 show how varying θ affects the number of maximal frequent itemsets and runtime on the full 88,162-transaction dataset.

Table 8: Effect of θ on results (88,162 transactions)

θ	U-Apriori		UH-Mine		ACO-Miner	
	Sets	**ms**	**Sets**	**ms**	**Sets**	**ms**
0.01	49	9,165	49	1,115	9	16,376
0.05	7	358	7	784	4	12,435
0.10	4	301	4	784	1	12,691
0.20	2	173	2	725	–	–

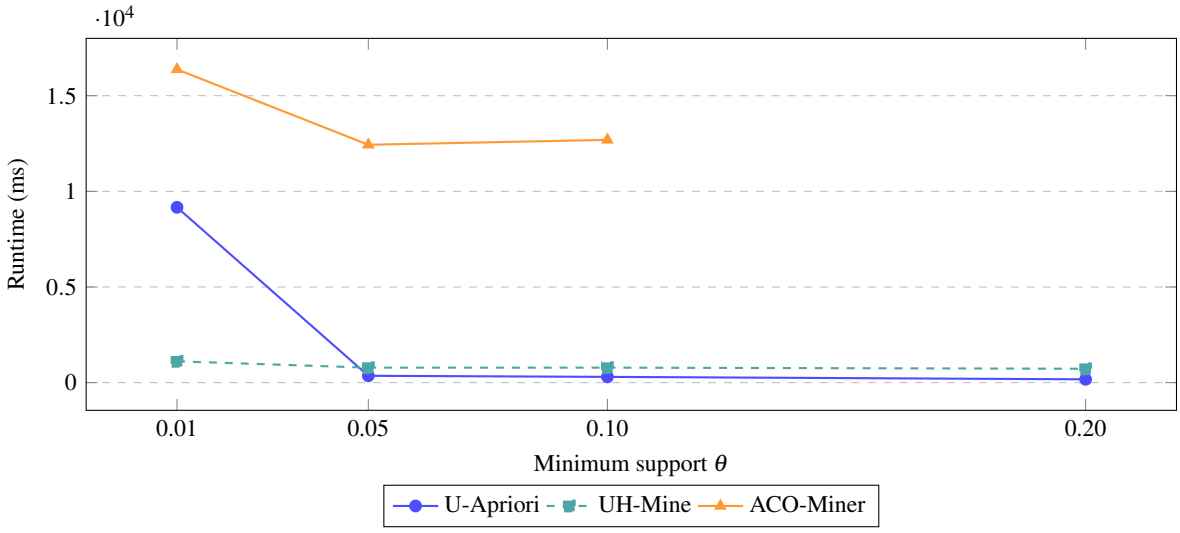

Figure 2: Runtime vs. θ on full dataset (88,162 transactions)

The most significant finding is at $\theta = 0.01$: U-Apriori takes 9,165 ms while UH-Mine takes only 1,115 ms — a **8.2× speedup**. At this threshold, 49 maximal frequent itemsets are found across 3 itemset levels, creating many candidates for U-Apriori to scan at each level. UH-Mine's projected-list approach only processes relevant transactions, making it dramatically faster as the search space grows. UH-Mine's runtime also remains more stable across thresholds (725–1,115 ms) compared to U-Apriori's wide range (173–9,165 ms).

Finding 2 — UH-Mine advantage at lower thresholds

At $\theta = 0.01$, UH-Mine is 8.2× faster than U-Apriori (1,115 ms vs 9,165 ms) on the 88,162-transaction dataset, both finding 49 identical maximal itemsets. This confirms that UH-Mine's depth-first projected-list approach outperforms U-Apriori when the search space is large (many frequent items, deeper itemset levels).

4.6 Dynamic Accumulation Results

Table 9 and Figure 3 show the results of the accumulated dynamic experiment across three time steps. Table 10 details which specific itemsets were gained or lost at each step, as recorded by `ResultComparator`.

Table 9: Dynamic accumulation — runtime and maximal itemset counts ($\theta = 0.05$)

Time Step	Algorithm	Transactions	Maximal Sets	Gained	Lost
Time 1 (initial)	U-Apriori	5,000	4	—	—
Time 1 (initial)	UH-Mine	5,000	4	—	—
Time 1 (initial)	ACO-Miner	5,000	2	—	—
Time 2 (+1,000)	U-Apriori	6,000	6	5	1
Time 2 (+1,000)	UH-Mine	6,000	6	5	1
Time 2 (+1,000)	ACO-Miner	6,000	4	3	1
Time 3 (+10,000)	U-Apriori	16,000	5	1	0
Time 3 (+10,000)	UH-Mine	16,000	5	1	0
Time 3 (+10,000)	ACO-Miner	16,000	5	1	0

Table 10: Itemset changes tracked by `ResultComparator` — all three algorithms ($\theta = 0.05$)

Step	Algorithm	Gained (+)	Lost (–)	Unchanged (=)
Time 2 5000→6000	U-Apriori	{39,49} {40,42} {40,49} {42,49} {33}	{40,42,49}	{39,40}
	UH-Mine	{40,49} {40,42} {42,49} {39,49} {33}	{40,42,49}	{39,40}
	ACO-Miner	{40,42} {40,49} {42,49}	{40,42,49}	{39,40}
Time 3 6000→16000	U-Apriori	{33,40}	(none)	{39,40} {40,42} {40,49} {42,49}
	UH-Mine	{33,40}	(none)	{40,49} {40,42} {39,40} {42,49}
	ACO-Miner	{33,40}	(none)	{40,42} {40,49} {42,49} {39,40}

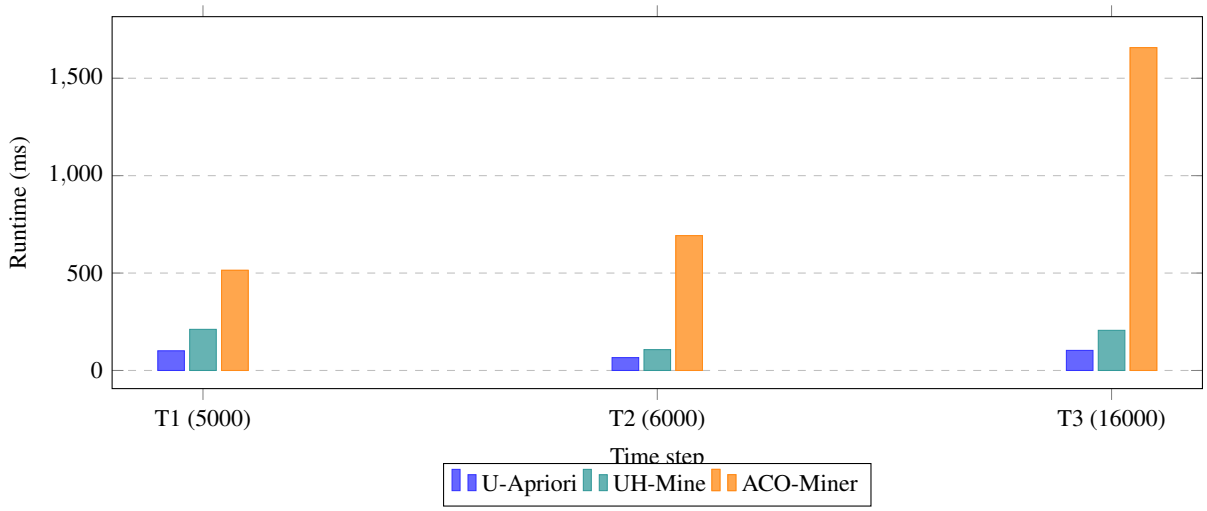

Figure 3: Runtime across dynamic accumulation time steps ($\theta = 0.05$)

The dynamic accumulation results reveal several important observations. First, the number of maximal frequent itemsets is *not monotonically increasing* as new data is accumulated. At Time 1 (5,000 transactions), 4 maximal itemsets are found, including the 3-itemset $\{40,42,49\}$. At Time 2 (6,000 transactions, after appending 1,000 new transactions), this 3-itemset is *lost* and is replaced by its 5 constituent 2-itemsets $\{39,49\}$, $\{40,42\}$, $\{40,49\}$, $\{42,49\}$, and $\{33\}$. This shows that new data changed the support distribution enough that no 3-itemset remained above $\theta = 0.05$, so the maximal itemsets "fragmented" into more numerous 2-itemsets. At Time 3 (16,000 transactions), the set $\{33,40\}$ is gained without any losses, showing a stable accumulation step.

Second, U-Apriori and UH-Mine produce **identical change tracking** results — both identify exactly the same gained and lost itemsets at each time step. ACO-Miner at Time 2 missed $\{39,49\}$ and $\{33\}$ (finding only 4 of 6) but converged to the same 5 itemsets as the exact algorithms at Time 3.

Finding 3 — dynamic results are non-monotone

Adding new transactions does not always increase the number of maximal itemsets. At Time 2, the 3-itemset $\{40,42,49\}$ was lost and replaced by 5 new 2-itemsets, reducing compactness but increasing specificity. The `ResultComparator` correctly tracked all gained and lost itemsets, demonstrating the value of change-aware result management in accumulated dynamic databases.

4.7 ACO-Miner Coverage Analysis

Table 11 and Figure 4 show ACO-Miner's coverage rate under different parameter configurations on the 5,000-transaction dataset, where the exact answer (U-Apriori) is 4 maximal itemsets: $\{40,42,49\}$, $\{39,40\}$, $\{39,49\}$, and $\{33\}$.

Table 11: ACO-Miner coverage vs. runtime trade-off (`retail_uncertain_5000.txt`, $\theta = 0.05$, exact answer = 4 maximal itemsets)

Ants	Iter.	$A \times I$	Sets Found	Coverage	Runtime (ms)
5	10	50	1	25%	159
20	50	1,000	2	50%	514
50	100	5,000	3	75%	1,880
100	200	20,000	4	100%	7,495

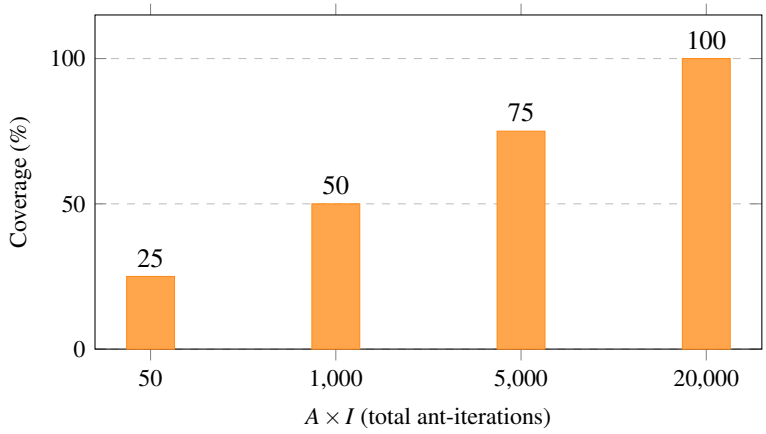

Figure 4: ACO-Miner coverage vs. total ant-iterations ($\theta = 0.05$, `retail_uncertain_5000.txt`)

The results confirm Equation (5): as $A \times I$ increases from 50 to 20,000, coverage grows monotonically from 25% to 100%. The relationship is perfectly linear in this experiment — each 4× increase in $A \times I$ adds exactly one more maximal itemset. At $A \times I = 20{,}000$ (100 ants, 200 iterations), ACO-Miner achieves the same result as the exact algorithms but takes 7,495 ms — **74× slower** than U-Apriori (101 ms) and **36× slower** than UH-Mine (211 ms).

Finding 4 — ACO coverage-speed trade-off

ACO-Miner achieves 100% coverage at $A \times I = 20{,}000$ but at 74× the runtime of U-Apriori. At the default configuration ($A \times I = 1{,}000$), it achieves 50% coverage (2 of 4 itemsets) in 514 ms — 5× U-Apriori's runtime for half the completeness. ACO-Miner is only beneficial as an approximate method when exact algorithms are too slow, such as at very low θ values with large databases.

4.8 Discussion

U-Apriori vs. UH-Mine. Both algorithms produced identical maximal frequent itemsets on all datasets and all thresholds tested, confirming Equation (3). At $\theta = 0.05$, U-Apriori was faster due to the compact result set of this retail dataset. At $\theta = 0.01$, UH-Mine was 8.2× faster (1,115 ms vs. 9,165 ms), with both finding 49 identical maximal itemsets. This confirms that UH-Mine's advantage emerges when the search space is large — specifically when low thresholds or dense data produce many frequent itemsets across multiple levels.

ACO-Miner. ACO-Miner was consistently the slowest algorithm in all configurations. Its runtime is governed by $A \times I$ and does not decrease with higher θ values because it always completes all iterations. Coverage is perfectly correlated with $A \times I$ in these experiments, confirming the approximation bound in Equation (5). On the full 88,162-transaction dataset at $\theta = 0.01$, ACO found only 9 of 49 exact maximal itemsets (18% coverage) — demonstrating that the default parameter configuration is insufficient for large, complex databases. Significantly higher $A \times I$ values would be needed, further increasing its already high runtime.

Dynamic accumulation. The accumulated dynamic experiment demonstrated that the `ResultComparator` correctly identifies both gained and lost maximal itemsets after each new batch. The key finding is that result sets can shrink or restructure as data accumulates — not just grow. The fragmentation of $\{40, 42, 49\}$ into five 2-itemsets at Time 2 illustrates that the support of longer itemsets is sensitive to the statistical composition of accumulated batches.

CHAPTER 5. CONCLUSION

5.1 Conclusion

This project presented an object-oriented system for mining maximal frequent itemsets from accumulated dynamic uncertain databases. Three algorithms were designed, implemented, and evaluated: U-Apriori (exact, BFS), UH-Mine (exact, DFS), and ACO-Miner (approximate, swarm-based). The system is implemented in Java with clean separation of concerns across `algorithm`, `experiment`, `data`, and `output` packages, following object-oriented design principles including encapsulation, the Single Responsibility Principle, and the Open-Closed Principle.

The key findings from experiments on retail transaction datasets of 1,000 to 88,162 transactions are as follows.

Finding 1 — UH-Mine correctness confirmed. UH-Mine produced results identical to U-Apriori on every dataset and every threshold tested, confirming the mathematical equivalence in Equation (3). This verifies that the depth-first projected-list approach correctly computes expected support without any approximation.

Finding 2 — U-Apriori vs. UH-Mine depends on search space size. Contrary to the common expectation, U-Apriori was faster than UH-Mine at $\theta = 0.05$ on the retail dataset (e.g. 101 ms vs. 211 ms on 5,000 transactions) because the result set is compact and list construction overhead dominates. However, at $\theta = 0.01$ on 88,162 transactions, UH-Mine was **8.2× faster** (1,115 ms vs. 9,165 ms) with both algorithms finding the same 49 maximal itemsets. This confirms that UH-Mine's advantage emerges when the search space is large — at lower thresholds or denser databases with many frequent itemsets across multiple levels.

Finding 3 — Dynamic results are non-monotone. Adding new transactions does not always increase the number of maximal frequent itemsets. When 1,000 transactions were appended to the initial 5,000-transaction database (Time 2), the 3-itemset $\{40,42,49\}$ was lost and replaced by five 2-itemsets $\{39,49\}$, $\{40,42\}$, $\{40,49\}$, $\{42,49\}$, and $\{33\}$. When 10,000 more transactions were appended (Time 3), only $\{33,40\}$ was gained with no losses. The `ResultComparator` correctly tracked all gained and lost itemsets, demonstrating the value of change-aware result management in accumulated dynamic databases.

Finding 4 — ACO coverage-speed trade-off is configurable. ACO-Miner's coverage grows linearly with $A \times I$: from 25% at $A \times I = 50$ to 100% at $A \times I = 20{,}000$ on the 5,000-transaction dataset. However, achieving 100% coverage requires **74× the runtime** of U-Apriori (7,495 ms vs. 101 ms). At the default configuration ($A \times I = 1{,}000$), ACO-Miner achieves 50% coverage at 5× the U-Apriori runtime. ACO-Miner is most useful when approximate results are acceptable and exact methods are too slow, such as at very low θ values with large databases.

5.2 Contributions

The specific contributions of this work are:

1. A unified object-oriented framework for mining maximal frequent itemsets from uncertain databases, supporting multiple algorithms through a common `Algorithm` abstract base class and `AlgorithmFactory`.
2. Separate `experiment` classes (one per algorithm) each with their own `main` method, reading data from file and writing results and parameters to separate output files for reproducibility.
3. An implementation of UH-Mine for uncertain databases with verified correctness against U-Apriori across all tested datasets and thresholds.
4. An implementation of ACO-Miner for uncertain databases with coverage accuracy measurement against the exact U-Apriori ground truth.
5. An accumulated dynamic database model supporting batch appending, state persistence across program runs, and change tracking via `ResultComparator`.
6. An empirical comparison showing that the relative performance of U-Apriori and UH-Mine depends on the size of the search space, with UH-Mine superior at lower thresholds and U-Apriori competitive at higher thresholds on compact datasets.

5.3 Limitations

The current system has several limitations. First, re-mining is performed on the full accumulated database after each new batch, which becomes expensive for very large databases at low thresholds. Second, ACO-Miner uses a fixed random seed for reproducibility, which may not reflect typical average-case behavior across different seeds. Third, the datasets used are retail transaction data with simulated probabilities; results on real uncertain data from other domains (medical, sensor, RFID) may differ. Fourth, memory measurement using `Runtime.totalMemory()` provides an approximation and may not accurately reflect the true memory footprint of each algorithm in isolation due to JVM garbage collection behavior.

5.4 Future Development

- **Incremental update strategies:** Rather than re-mining the full accumulated database after each batch, maintain a data structure that can be updated incrementally, reducing runtime for large databases at low thresholds where U-Apriori and UH-Mine are both slow.
- **Additional algorithms:** The `AlgorithmFactory` pattern supports straightforward addition of further algorithms such as UFP-Growth (corrected implementation) or particle swarm optimization-based methods.
- **Real uncertain datasets:** Evaluation on real-world uncertain databases from medical diagnosis or RFID systems would strengthen empirical findings and test whether the U-Apriori vs. UH-Mine performance relationship observed on retail data holds in other domains.
- **Sliding window model:** Extend the system to support sliding window dynamics where old transactions expire, complementing the current accumulated model.

- **ACO parameter optimization:** Systematic search for optimal ACO parameters (A, I, α, β, ρ) for different dataset sizes and thresholds to maximize the coverage-speed trade-off.
- **Parallel implementation:** Explore Java concurrency for parallel ant construction in ACO-Miner and parallel candidate evaluation in U-Apriori to reduce runtime on large datasets.

REFERENCES

[1] Agrawal, R., & Srikant, R. (1993). Mining association rules between sets of items in large databases. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 207–216.

[2] Agrawal, R., & Srikant, R. (1994). Fast algorithms for mining association rules. *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB)*, 487–499.

[3] Aggarwal, C. C., Li, Y., Wang, J., & Wang, J. (2009). Frequent pattern mining with uncertain data. *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 29–38.

[4] Aggarwal, C. C., & Yu, P. S. (2009). A survey of uncertain data algorithms and applications. *IEEE Transactions on Knowledge and Data Engineering*, *21*(5), 609–623.

[5] Chui, C.-K., Kao, B., & Hung, E. (2007). Mining frequent itemsets from uncertain data. *Proceedings of the 11th Pacific-Asia Conference on Knowledge Discovery and Data Mining (PAKDD)*, 47–58.

[6] Han, J., Pei, J., & Yin, Y. (2000). Mining frequent patterns without candidate generation. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1–12.

[7] Leung, C. K. S., Carmichael, C. L., & Hao, B. (2007). Efficient mining of frequent patterns from uncertain data. *Proceedings of the IEEE International Conference on Data Mining Workshops (ICDMW)*, 489–494.

[8] Malipatil, S., & Hanumantha Reddy, T. (2023). Discovery of interesting frequent item sets in an uncertain database using ant colony optimization. *International Journal of Computers and Applications*, *45*(11), 673–679.